

Universidad EAFIT
Departamento de Informática y Sistemas
Curso: Sistemas de Información — Proyecto Integrador 1 (ST0251)

TALLER 3

INTEGRACIÓN DE IA EN EL SISTEMA DE RECOMENDACIÓN (Ureview / PupiGo)

Integrante:
Miguel Ángel Colorado Castaño

Materia:
Proyecto Integrador 1

Video de demostración:
<https://youtu.be/01Ss10SWZ38>

Universidad EAFIT
Escuela de Ciencias Aplicadas e Ingeniería
Medellín
2026

ÍNDICE

1. Introducción

2. Enlaces del repositorio y trazabilidad

- 2.1 Repositorio del trabajo (enunciado original)
- 2.2 Repositorio fork (implementación propia)
- 2.3 Rama de la solución
- 2.4 Decisión de proveedores de IA

3. Evidencias de ejecución por requerimiento

- 3.1 Modificación de la descripción de la primera película (update_descriptions.py)
- 3.2 Actualización de descripciones vía CSV (update_movies_from_csv)
- 3.3 Modificación de la imagen de la primera película (update_images.py)
- 3.4 Modificación de todas las imágenes (update_images_from_folder)
- 3.5 Generación de embeddings y similitud de coseno (movie_embeddings / movie_similarities)
- 3.6 Embedding de una película al azar (show_random_embedding)

4. Trazabilidad de costos y uso de API

5. Conclusiones

6. Bibliografía / Referencias técnicas

7. Video de demostración

1. INTRODUCCIÓN

Este informe documenta la evidencia de ejecución del Taller 3, correspondiente a la integración de herramientas de inteligencia artificial dentro del proyecto Ureview/PupiGo (Django). El taller original propone el uso de la API de OpenAI para dos frentes: (a) el enriquecimiento de las sinopsis de las películas y (b) la generación de embeddings para el sistema de recomendación. En esta implementación se sustituyeron ambos proveedores manteniendo el mismo patrón exigido por el enunciado (comando con corte para una sola película + proceso masivo aparte), documentado en la sección 2.4.

Cada sección de evidencia (numeral 3) incluye el comando ejecutado, la captura de la terminal con su salida y, cuando aplica, la captura del panel de administración de Django que confirma el cambio persistido en base de datos.

2. ENLACES DEL REPOSITORIO Y TRAZABILIDAD

2.1 Repositorio del trabajo (enunciado original)

https://github.com/jdmartinev/TallerIA_PI.git

2.2 Repositorio fork (implementación propia)

https://github.com/migueCOLORADO/TallerIA_PI.git

2.3 Rama de la solución

miguel-colorado — rama donde se implementó y verificó la totalidad de los cambios descritos en este documento.

2.4 Decisión de proveedores de IA

- Texto (sinopsis): OpenAI → sustituido por **Claude** (Anthropic API, modelo `claude-haiku-4-5`). Se usa en un único comando (`update_descriptions`) que procesa la primera película, replicando el patrón del taller ("`break`" tras el primer registro).
- Embeddings / recomendación: OpenAI → sustituido por un modelo local de **sentence-transformers** (`intfloat/multilingual-e5-small`), sin costo por consulta, sin API key y sin envío de datos a terceros.

3. EVIDENCIAS DE EJECUCIÓN POR REQUERIMIENTO

3.1 Modificación de la descripción de la primera película (`update_descriptions.py`)

Comando ejecutado:

```
python manage.py update_descriptions
```

Figura 3.1a — Terminal: sinopsis original vs. sinopsis enriquecida por Claude para "Carmencita".

```
(venv) PS C:\Users\miguel\Proyecto_Integrador_1\Talleres\TallerIA_PI> python manage.py update_descriptions
Found 148 movies
Processing: Carmencita
Original synopsis: Carmencita documenta los albores del cine mediante el registro de una bailarina española ejecutando una danza tradicional apasionada. Ataviada elegantemente con vestido de volantes y zapatos blancos, la artista despliega giros y patadas con maestría, irradiando vitalidad teatral. Este breve pero histórico fragmento captura la gracia del movimiento humano, preservando la esencia del espectáculo decimonónico y demostrando las posibilidades expresivas del nuevo medio cinematográfico.
Updated synopsis: Carmencita documenta los primeros pasos del cinematógrafo a través de una bailarina española que ejecuta una danza tradicional con apasionada maestría. Ataviada con elegante vestido de volantes, la artista despliega giros y patadas que irradian vitalidad teatral. Este fragmento histórico captura la gracia del movimiento humano, demostrando las revolucionarias posibilidades expresivas del nuevo medio y preservando la esencia del espectáculo decimonónico en celuloide.
Updated: Carmencita
```

Figura 3.1b — Admin de Django: sinopsis de "Carmencita" persistida en base de datos, coincide con la salida de terminal.

The screenshot shows the Django administration interface for a movie database. The left sidebar contains navigation links for 'AUTHENTICATION AND AUTHORIZATION' (Groups, Users), 'MOVIE' (Movies, Reactions, Reviews), and 'NEWS' (News). The main content area is titled 'Change movie' and shows the details for the movie 'Carmencita'. The synopsis field is populated with the updated synopsis from the terminal output. Other fields include Release year (1894), Genre (Documentary, Short), Poster (/media/movie/images/m_carmencita.png), Director (William K.L. Dickson), Duration (1), and Content type (Película). Buttons for 'SAVE', 'Save and add another', 'Save and continue editing', and 'Delete' are at the bottom.

3.2 Actualización de descripciones vía CSV (update_movies_from_csv)

Comando ejecutado:

```
python manage.py update_movies_from_csv
```

Figura 3.2 — Terminal: número de películas actualizadas desde updated_movie_descriptions.csv.

```
(venv) PS C:\Users\miguel\Proyecto_Integrador_1\Talleres\TallerIA_PI> python manage.py update_movies_from_csv
Películas actualizadas: 148
```

3.3 Modificación de la imagen de la primera película (update_images.py)

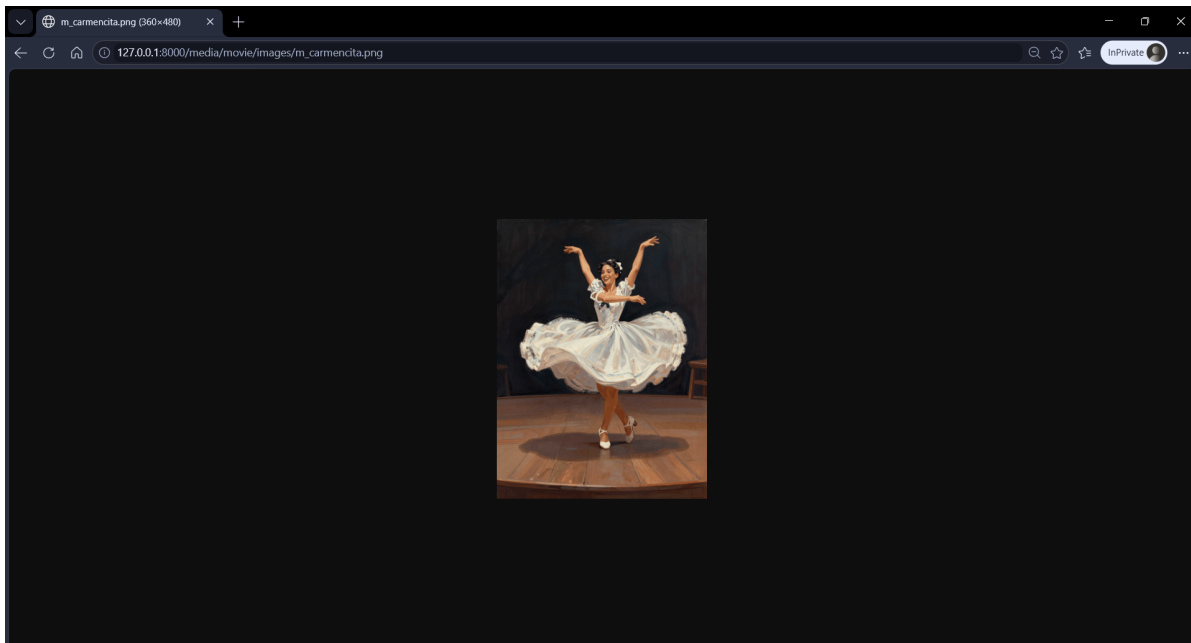
Comando ejecutado:

```
python manage.py update_images
```

Figura 3.3a — Terminal: descarga del póster generado (Higgsfield) y asignación a la primera película.

```
(venv) PS C:\Users\miguel\Proyecto_Integrador_1\Talleres\TallerIA_PI> python manage.py update_images
Found 148 movies
Higgsfield job c194b1e7-2524-48a5-8f3f-f4e9a8dff618 (z_image)
Saved and updated image for: Carmencita -> /media/movie/images/m_carmencita.png
Process finished (only first movie updated).
```

Figura 3.3b — Admin de Django: póster de "Carmencita" actualizado (m_carmencita.png), visible en el campo Poster.



3.4 Modificación de todas las imágenes (update_images_from_folder)

Comando ejecutado:

```
python manage.py update_images_from_folder
```

Figura 3.4 — Terminal: pósters asignados en bloque desde media/movie/images/, con conteo de archivos procesados.

```
(venv) PS C:\Users\miguel\Proyecto Integrador 1\Talleres\TallerIA_PI> python manage.py update_images_from_folder
Pósters asignados: 62 (archivos en carpeta: 62)
Películas sin archivo de póster (86): A Beast at Bay, A Corner in Wheat, A Fantasy, A Terrible Night, A Trip to the Moon, A Turn of t
he Century Illusionist, Alice in Wonderland, Cabiria, Cinderella, Cleopatra, Dante's Inferno, Death's Marathon, Dream of a Rarebit Fi
end, Ella Lola, a la Trilby, Fairyland: A Kingdom of Fairies, Fantomas, Fantomas Against Fantomas, Faust et Møphistophøls, Frankenst
ein, Gertie the Dinosaur, Grandma's Reading Glass, Her Crowning Glory, His Musical Career, His New Profession, Humorous Phases of Fun
ny Faces, Hypocrites, In the Border States, In the Land of the Head Hunters, Ingeborg Holm, Jack and the Beanstalk, Judith of Bethuli
a, Juve Against Fantomas, Laughing Gas, Le faux magistrat, Let Me Dream Again, Life of an American Fireman, Mabel at the Wheel, Mabel
's Married Life, Making a Living, New York Subway, Panorama of Esplanade by Night, Rescued by Rover, The '?' Motorist, The 400 Tricks
of the Devil, The Avenging Conscience: or 'Thou Shalt Not Kill', The Battle at Elderbush Gulch, The Bewitched Inn, The Burglar's Dil
emma, The Conquest of the Pole, The Country Doctor, The Dead Man Who Killed, The Electric Hotel, The Great Train Robbery, The House o
f Darkness, The India Rubber Head, The Inside of the White Slave Traffic, The Knockout, The Land Beyond the Sunset, The Last Days of
Pompeii, The Life and Death of King Richard III, The Lonedale Operator, The Lonely Villa, The Mender of Nets, The Miser's Heart, The
Music Lover, The Musketeers of Pig Alley, The Mysterious X, The New York Hat, The Ocean Waif, The Painted Lady, The Red Spectre, The
Revenge of a Kinematograph Cameraman, The Sealed Room, The Sick Kitten, The Sign of the Cross, The Story of the Kelly Gang, The Stude
nt of Prague, The Unchanging Sea, The Voyage Across the Impossible, The Woman Always Pays, The Wonderful Wizard of Oz, Those Awful Ha
ts, Traffic in Souls, Trødgørdsmøstaren, Une partie de cartes, Winsor McCay, the Famous Cartoonist of the N.Y. Herald and His Moving
Comics
```

3.5 Generación de embeddings y similitud de coseno

Comando ejecutado:

```
python manage.py movie_embeddings python manage.py movie_similarities
```

El comando movie_embeddings genera y persiste los vectores; movie_similarities los reutiliza para calcular similitud coseno entre películas y contra un prompt en lenguaje natural.

Figura 3.5a — Terminal: generación y almacenamiento del embedding (campo emb) para todo el catálogo.

```
(venv) PS C:\Users\miguel\Proyecto_Integrador_1\Talleres\TallerIA_PI> python manage.py movie_embeddings
[✓] Embedding stored for: Toy Story
[✓] Embedding stored for: Get Out
[✓] Embedding stored for: The Conjuring
[✓] Embedding stored for: La La Land
[✓] Embedding stored for: Parasite
[✓] Embedding stored for: Mad Max: Fury Road
[✓] Embedding stored for: Gladiator
[✓] Embedding stored for: The Grand Budapest Hotel
[✓] Embedding stored for: Superbad
[✓] Embedding stored for: Coco
[✓] Embedding stored for: Spirited Away
[✓] Embedding stored for: Whiplash
[✓] Embedding stored for: Se7en
[✓] Embedding stored for: No Country for Old Men
[✓] Embedding stored for: The Avengers
[✓] Embedding stored for: Spider-Man: Into the Spider-Verse
[✓] Embedding stored for: Coraline
[✓] Embedding stored for: Her
[✓] Embedding stored for: Breaking Bad
[✓] Embedding stored for: Game of Thrones
[✓] Embedding stored for: Stranger Things
[✓] Embedding stored for: The Office
[✓] Embedding stored for: Friends
[✓] Embedding stored for: The Wire
[✓] Embedding stored for: Sherlock
[✓] Embedding stored for: The Crown
[✓] Embedding stored for: Black Mirror
[✓] Embedding stored for: The Mandalorian
[✓] Embedding stored for: Better Call Saul
[✓] Embedding stored for: Dark
[✓] Embedding stored for: Peaky Blinders
[✓] Embedding stored for: The Simpsons
[✓] Embedding stored for: Rick and Morty
[✓] Embedding stored for: Narcos
[✓] Embedding stored for: Money Heist
[✓] Embedding stored for: Chernobyl
[✓] Embedding stored for: Fargo
[✓] Embedding stored for: Avatar: The Last Airbender
[✓] Finished generating embeddings for 148 movies (dimensión 384)
```

Figura 3.5b — Terminal: similitud de coseno entre dos películas y entre un prompt de texto y cada una.

```
(venv) PS C:\Users\miguel\Proyecto_Integrador_1\Talleres\TallerIA_PI> python manage.py movie_similarities
Modelo de embeddings: intfloat/multilingual-e5-small (local)
Película 1: Interstellar (Ciencia Ficción)
Película 2: The Godfather (Crimen)
Prompt: "película de ciencia ficción"
[📄] Similaridad entre 'Interstellar' y 'The Godfather': 0.7989
[📄] Similitud prompt vs 'Interstellar': 0.8787
[📄] Similitud prompt vs 'The Godfather': 0.7926
```

3.6 Embedding de una película al azar (show_random_embedding)

Comando ejecutado:

```
python manage.py show_random_embedding
```

Figura 3.6 — Terminal: dimensión, dtype, norma y primeros valores del embedding almacenado.

```
(venv) PS C:\Users\miguel\Proyecto_Integrador_1\Talleres\TallerIA_PI> python manage.py show_random_embedding
[📄] Película: Money Heist (2017, Crimen)
Sinopsis: Un profesor ingeniero reúne a un grupo de delincuentes especializados para ejecutar el robo más ambicioso jamás concebido: infiltrarse en la Casa de la Moneda e...
Dimensión del embedding: 384 | dtype: float32 | norma: 1.0000
Primeros 10 valores: [ 0.0683 -0.04588 -0.07912 -0.05801 0.06791 -0.10465 0.03034 0.02496
 0.07887 0.01802]
```

4. TRAZABILIDAD DE COSTOS Y USO DE API

Con el fin de dejar explícita la huella de consumo de API externa del proyecto:

- El campo ANTHROPIC_API_KEY (.env) se usa exclusivamente dentro de update_descriptions.py, con una sola llamada por ejecución (procesa solo la primera película).
- El endpoint /recommend/ de la aplicación **no** consume la API de Claude: usa el modelo local de embeddings ya cargado en memoria del proceso de Django.

- Modelo empleado: `claude-haiku-4-5` (el más económico de la línea Claude), con un tope de salida de 150 tokens por sinopsis, lo que mantiene el costo por ejecución en el orden de centésimas de centavo de dólar.

5. CONCLUSIONES

La sustitución de OpenAI por Claude (texto) y por un modelo local de sentence-transformers (embeddings) permitió cumplir el objetivo del taller — integrar IA para generar y dinamizar el contenido del proyecto — sin incurrir en costos recurrentes por cada recomendación servida a los usuarios finales, y concentrando el único gasto de API en un paso puntual y acotado del pipeline de datos (enriquecimiento de sinopsis).

6. BIBLIOGRAFÍA / REFERENCIAS TÉCNICAS

Anthropic. Claude API Documentation. <https://docs.claude.com>

Hugging Face / sentence-transformers. `intfloat/multilingual-e5-small`.
<https://huggingface.co/intfloat/multilingual-e5-small>

Higgsfield. Documentación de generación de imágenes. <https://higgsfield.ai>

Django Software Foundation. Django Documentation. <https://docs.djangoproject.com>

Materiales del curso ST0251 — Proyecto Integrador 1, Universidad EAFIT.

7. VIDEO DE DEMOSTRACIÓN

Recorrido en video por la aplicación y la ejecución de los comandos del taller:

<https://youtu.be/01Ss10SWZ38>